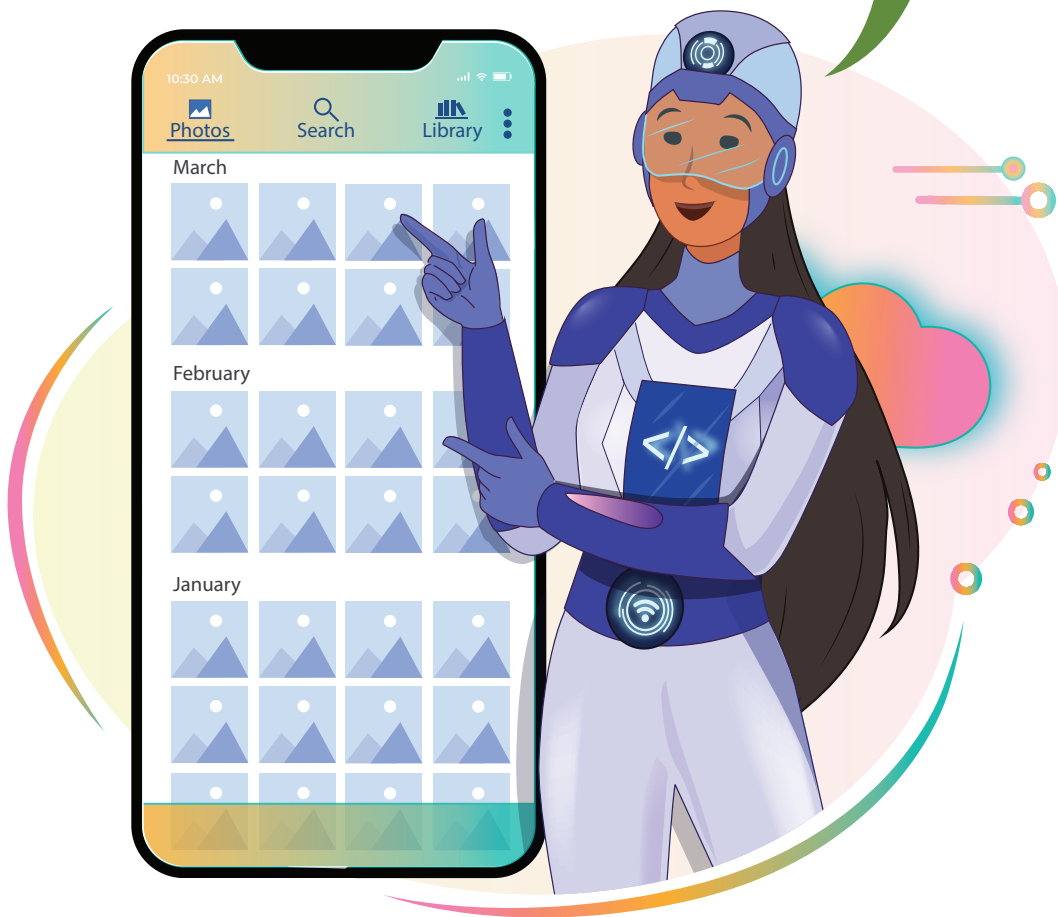




Bubble Sort

08

Smartphone galleries sort pictures according to their dates so they can be easily found. Digital devices use algorithms to automatically sort data inside them.



We have seen what algorithms are and how they work. We use logical thinking and similar concepts in many parts of our daily lives too. We tend to sort items according to their price while shopping online. Words in a dictionary are sorted in an alphabetical order too.

8. Bubble Sort

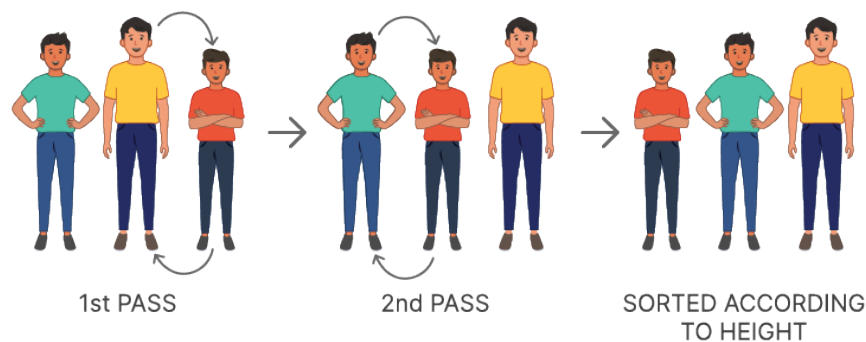
Sorting

Sorting is the process that arranges things in a particular order. It makes it easier to access things.

For example, consider a few children standing in a row for school prayers. The teacher arrives and tells them to stand in an increasing order according to their height. How would they sort themselves?

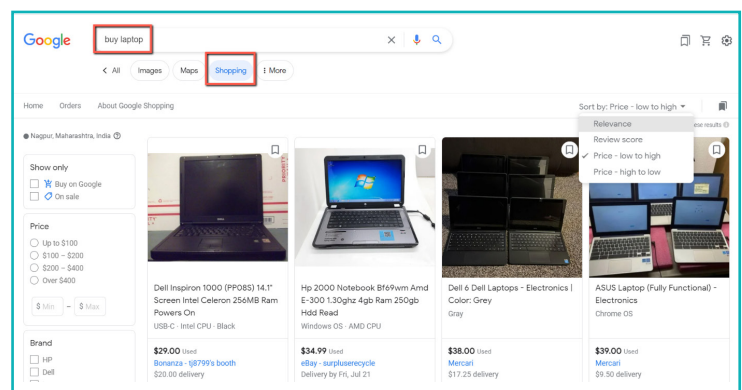


Each student can compare his/her height with the student behind and in front and swap places if required. This will sort them into a height-wise line.



Sorting is not limited to our daily work only; it is also an important concept in programming. Every day, we use our eyes, hands, and common sense to sort elements, but in programming we use algorithms to sort data.

If we search online for “buy laptop”, it displays multiple options. We can sort them by price or any other parameter. All websites and applications implement sort algorithms to display results.



Activity

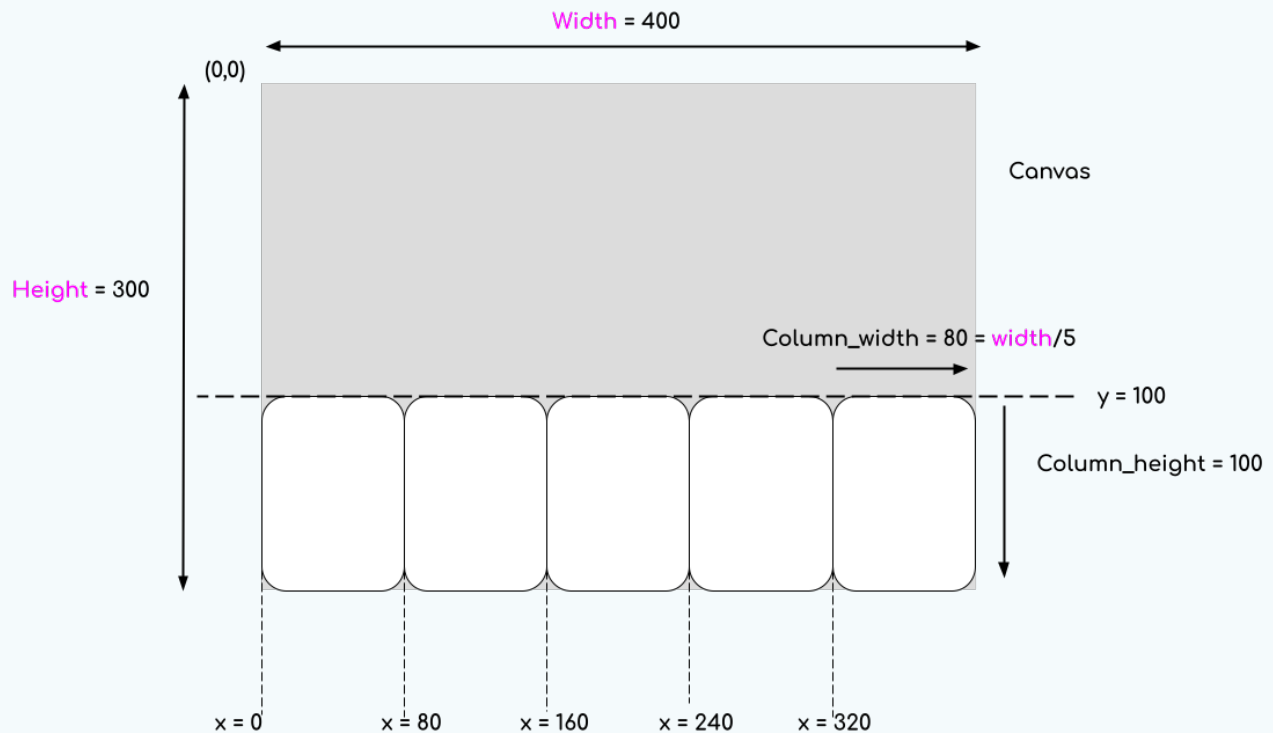
Sorting algorithm and implement it to sort columns with random heights in an ascending order using array and functions.

1 Open p5.js editor

- Rename the project as “Sorting”.
- Change the size of the canvas to 400x300.

2 Draw Columns

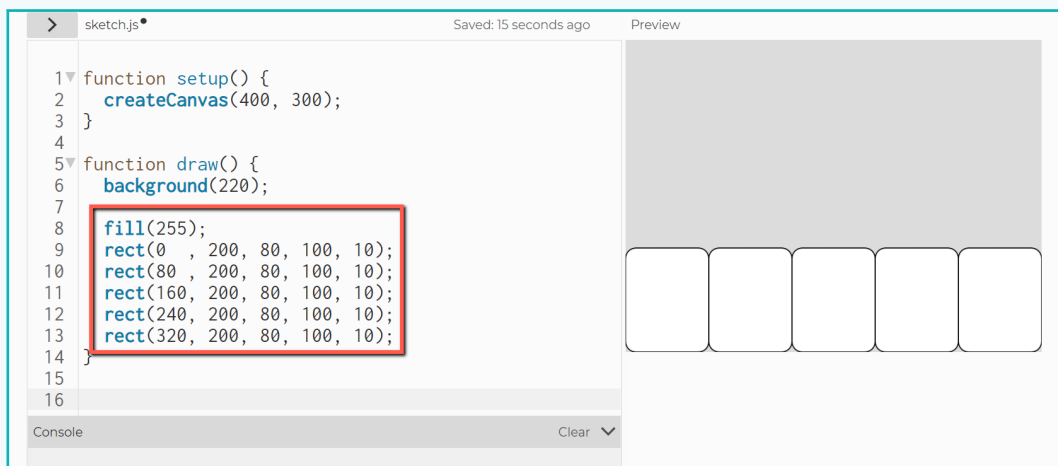
To draw multiple columns, use `rect()` command for each column.



Activity

3 Adding Columns

- Add fill() command to fill white colour in the shape.
- Add rect(x, y, width, height, corner_radius); command to draw rectangular columns with height = 100. Set the width such that 5 columns will fit on the canvas. Keep corner radius=10 for the rectangles.

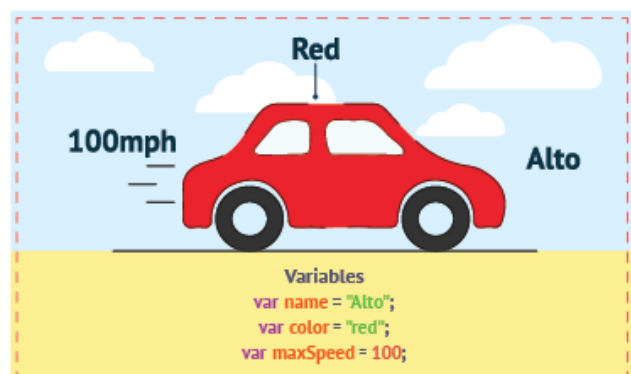


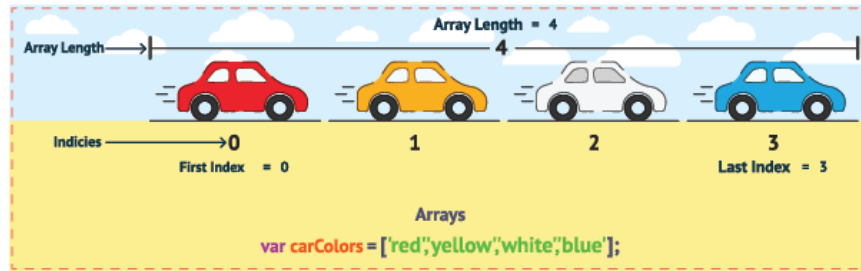
Here, if we need to add or delete columns or wish to change column properties, it will become tedious to make many changes in each rect() command. Also, to create multiple number of columns, we will repeat the code. We can avoid repetitive coding and solve this problem using Arrays.

Array

An array is a type of data that holds multiple values. We can access values in an array through their positions in a list. We can also add, remove, or modify values in the array.

For example, consider a car. Information related to the car can be saved in variables like name, colour etc.





Note that each value in the array has an index and we can access its specific value using the index. For every array, the index starts with 0. The last index of an array is always one less than the total number of array elements. Here, we have 4 elements in an array named `carColors`, the indices are from 0 to 3.

We can declare an empty array at the beginning of the code before `start()` function.

```
var columns = [ ];
```

Add array values in `setup()` function.

```
var columns = [100, 200, 100, 300, 150];
```

This is an array of 5 values. 0th index element has value 100, 1st index element has value 200 and so on. We can access each element of an array individually using the array name and index number of the specific element. The second element in the array has index 1 and can be accessed as `column[1]` which is equal to 200.

Activity

We will also create an array of heights of the columns which can be sorted in ascending order of height later.

- Declare an array variable called 'columns' at the beginning of the code to store column height.
- We can directly assign the values of array in `setup()`.



Activity

Instead of using height value 100 in rect() command, we can insert a value from columns array i.e. columns[0] to columns[4] in rect() command.

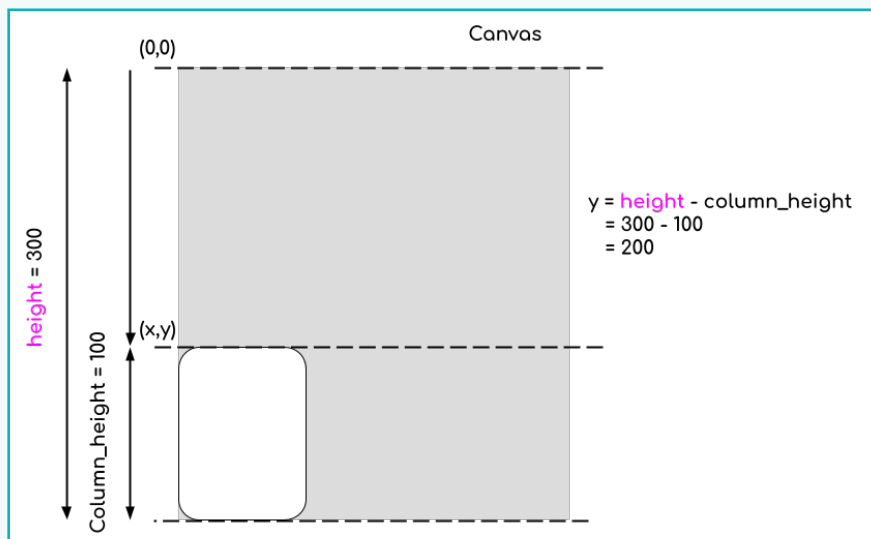
- Use array values in columns array to set height in rect() command.

```

> sketch.js Saved: 15 seconds ago Preview
1 var columns = [];
2
3 function setup() {
4   createCanvas(400, 300);
5
6   columns = [100,100,100,100,100];
7 }
8
9 function draw() {
10  background(220);
11
12  fill(255);
13  rect(0 , 200, 80, columns[0], 10);
14  rect(80 , 200, 80, columns[1], 10);
15  rect(160, 200, 80, columns[2], 10);
16  rect(240, 200, 80, columns[3], 10);
17  rect(320, 200, 80, columns[4], 10);
18 }

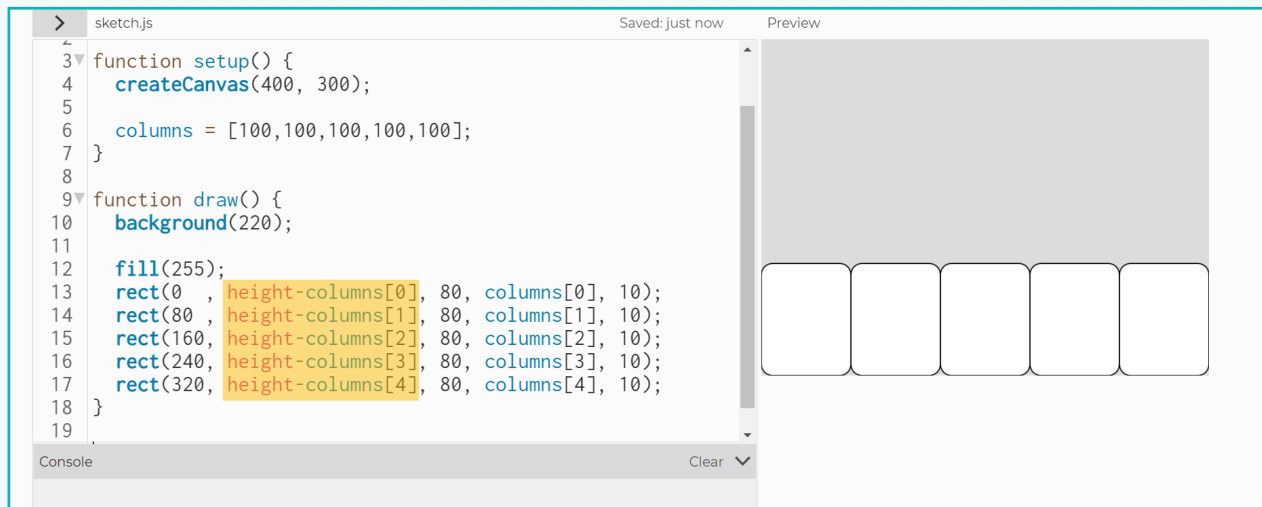
```

We can also determine Y position of each rectangle by the rectangle height and canvas height i.e. using array values.



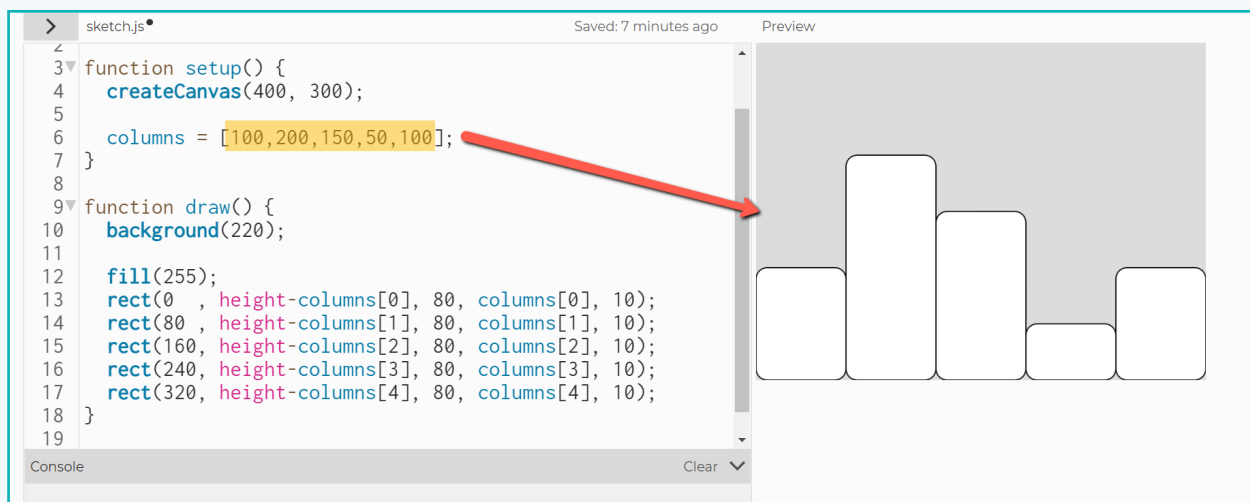
Activity

- Replace value 200 (for y position) in each rect() command using canvas and column height values in columns[] array. For example (height – columns[0]) for first column.



```
3 function setup() {
4   createCanvas(400, 300);
5
6   columns = [100, 100, 100, 100, 100];
7 }
8
9 function draw() {
10  background(220);
11
12  fill(255);
13  rect(0, height-columns[0], 80, columns[0], 10);
14  rect(80, height-columns[1], 80, columns[1], 10);
15  rect(160, height-columns[2], 80, columns[2], 10);
16  rect(240, height-columns[3], 80, columns[3], 10);
17  rect(320, height-columns[4], 80, columns[4], 10);
18 }
19
```

Even if we change the height values in the array now, the columns will be placed along the bottom line of the canvas.



```
3 function setup() {
4   createCanvas(400, 300);
5
6   columns = [100, 200, 150, 50, 100];
7 }
8
9 function draw() {
10  background(220);
11
12  fill(255);
13  rect(0, height-columns[0], 80, columns[0], 10);
14  rect(80, height-columns[1], 80, columns[1], 10);
15  rect(160, height-columns[2], 80, columns[2], 10);
16  rect(240, height-columns[3], 80, columns[3], 10);
17  rect(320, height-columns[4], 80, columns[4], 10);
18 }
19
```

Exercise

Answer in one line

Here is an array of different fruits

```
var myFruits = ['mango', 'banana', 'grapes', 'orange', 'cherry'];
```

1 How many elements are in the array?

▲ _____

2 What will be the value of **myFruits[3]**?

▲ _____

3 How can we access value '**mango**' from the array?

▲ _____

Activity

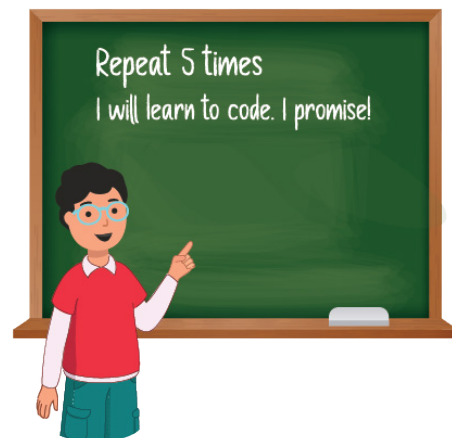
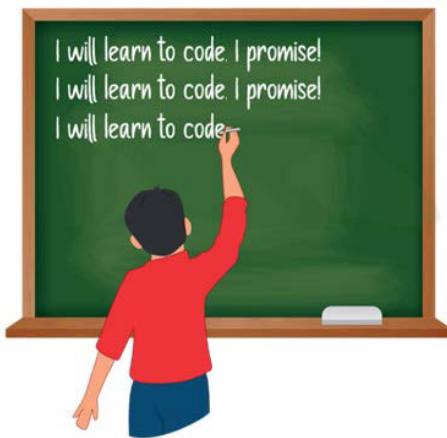
If we want to add 10 columns instead of 5, it is tedious to write `rect()` command 10 times. If any changes need to be made, we have to make them in all 10 `rect()` commands.

Instead of writing `rect()` command 10 times to draw 10 columns, we can use a 'For Loop'.

In programming, if an action needs to be repeated multiple times, a loop code structure is used that tells the computer to repeat a specific code block a specified number of times.

Loops

A loop is a programming structure that repeats a sequence of instructions multiple times or till a specific condition is met.



Loops help to keep the code short, lessen errors and save programming time.

Loops are of different types. One of them is a “For Loop”.

For Loop

A for loop is a code statement used to declare an iteration that repeats a set of code instructions. These instructions are to be executed a specific number of times. We often use this while working with arrays.

Example: For loops are typically used when the number of iterations is known before entering the loop. The code structure is as shown.

```
for(statement1;statement2;statement3){  
    //code block to be executed  
}
```

Statement 1 is used to initialize counter and is evaluated once, before the loop begins. It is usually an expression, or a variable declaration typically used to initialise a counter variable.

Example: Counter can have any initial value but as array index starts from 0, we can start the loop counter from 0 in our case.

```
for (let i=0; statement2; statement3) {  
    //code block to be executed  
}
```

For this expression, we declare a new variable with ‘let’ keyword and assign an initial value.

This executes once before the execution of the code block.

Note:

In this case, 'let' keyword is used to declare a variable. This is used to declare a variable instead of 'var' when the scope of the variable is limited to the code block.

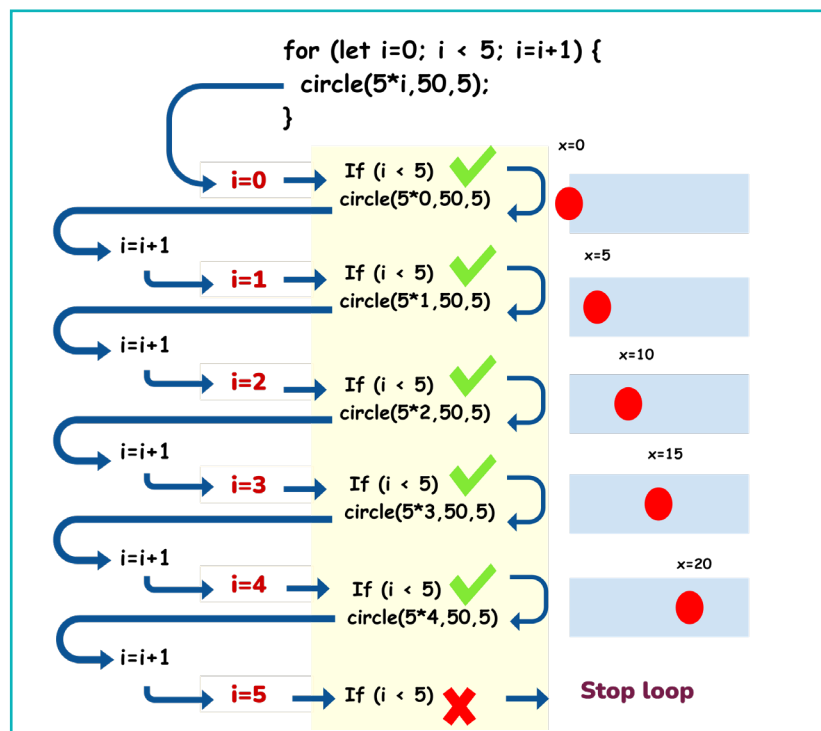
Statement 2 is a condition that is checked before each loop iteration. If this expression evaluates to true, the code block inside the loop is executed. If the condition evaluates to false, the loop stops.

For example: Since we want to draw 5 columns, we want to repeat the command 5 times. The condition for the repetition will be ' $i < 5$ '. As the value of i is starting from 0, the counter will be increment to 4 i.e. less than 5.

```
for(let i=0; i<5; statement3){
  // code block to be executed
}
```

Statement 3 is the final expression that is evaluated at the end of each loop iteration. It is generally used to update or increase the counter variable.

For example: Here, we want to use i as a counter that should increment by 1 every time the loop runs.



Instead of writing $i < 5$, we can find the actual length of an array with ".length" operator. If we add '.length' to the name of an array, it gives the number of elements present in an array.

Activity

- c. Try typing `console.log(columns.length)` in the code. You will be able to see '5' in the console as the length of the array. Try adding another element in the array and check the console for the log.



```
1 var columns = [];  
2  
3 function setup() {  
4   createCanvas(400, 300);  
5  
6   columns = [100,200,150,50,100];  
7 }  
8  
9 function draw() {  
10  
11   console.log(columns.length);  
12  
13   background(220);  
14 }  
15
```

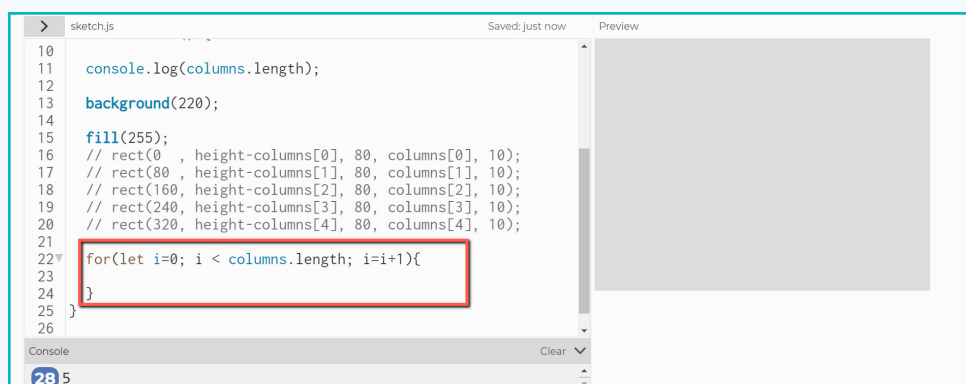
Console

```
27 5  
27 5  
27 5
```

4 Drawing Columns using For Loops

Use for loop to draw columns

- Comment out or delete all 5 `rect()` commands added earlier.
- Add for loop with '`i=0`', with condition as "`i < columns.length`" and increment '`i`' by 1.



```
10  
11 console.log(columns.length);  
12  
13 background(220);  
14  
15 fill(255);  
16 // rect(0 , height-columns[0], 80, columns[0], 10);  
17 // rect(80 , height-columns[1], 80, columns[1], 10);  
18 // rect(160, height-columns[2], 80, columns[2], 10);  
19 // rect(240, height-columns[3], 80, columns[3], 10);  
20 // rect(320, height-columns[4], 80, columns[4], 10);  
21  
22 for(let i=0; i < columns.length; i=i+1){  
23  
24 }  
25 }  
26
```

Console

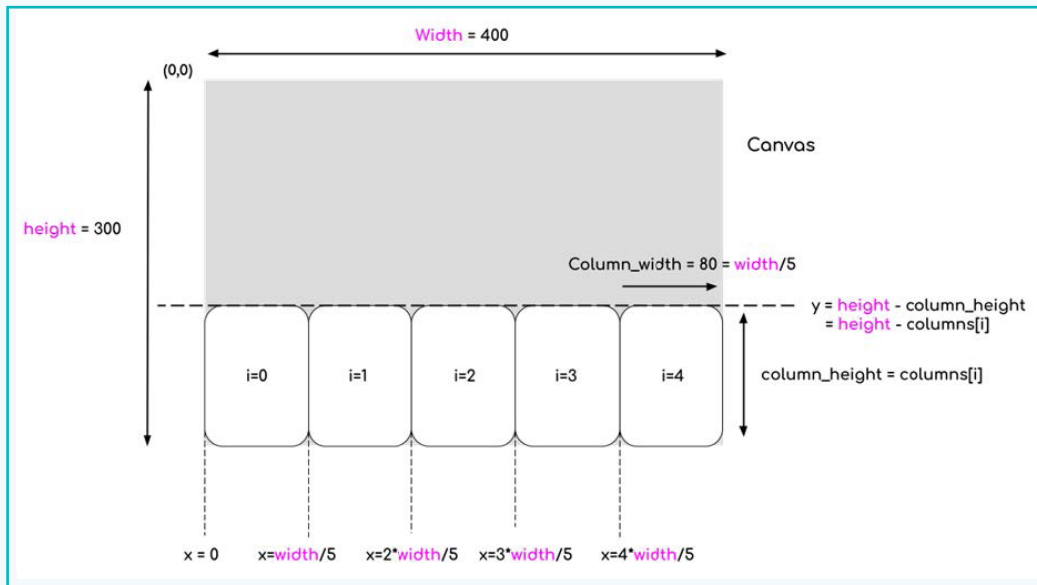
```
28 5
```

Inside for loop, we can add `rect()` command that uses '`i`' as incrementor to draw columns as shown in the figure.

Note:

Width and height are default variables in p5.js. They have the values of the canvas' width and height.

Activity



As seen in the diagram above, width of all the columns is same and can be calculated as
 Width of the column = canvas width/Number of columns
 = width/columns.length

This can also be used to determine x position of each column. First column will be at $x=0$, second at $x = \text{width}/\text{columns.length}$, third at $2 * \text{width}/\text{columns.length}$ and so on. Hence, we can use counter 'i' in loop to determine x position.

- c. Add rect() command with column_width = width/(columns.length), column_height = columns[i] at position $x = i * \text{width}/(\text{columns.length})$ and $y = \text{height} - \text{columns}[i]$ with corner radius=10.

```
for(let i=0; i < columns.length; i=i+1){
    rect(i*width/columns.length, height-columns[i],width/columns.length,columns[i],10);
}
```

To make it simpler, we can create a variable to store the length of an array as a count.

```
sketch.js
Saved: just now

1 var columns = [];
2 var count;
3
4 function setup() {
5     createCanvas(400, 300);
6
7     columns = [100,200,150,50,100];
8
9     count = columns.length
10 }
11
12 function draw() {
13
14     console.log(columns.length);
15
16     background(220);
17
18     fill(255);
```

Activity

In the 'for loop', replace **columns.length** in rect() command with 'count' variable.

```

13
14 console.log(columns.length);
15
16 background(220);
17
18 fill(255);
19 // rect(0, height-columns[0], 80, columns[0], 10);
20 // rect(80, height-columns[1], 80, columns[1], 10);
21 // rect(160, height-columns[2], 80, columns[2], 10);
22 // rect(240, height-columns[3], 80, columns[3], 10);
23 // rect(320, height-columns[4], 80, columns[4], 10);
24
25 for(let i=0; i < columns.length; i=i+1){
26
27   rect(i*width/count, height-columns[i], width/count, columns[i], 10);
28
29 }
30
31

```

We have created columns using a single rect() command and a for loop. This way, we avoided the repetitive use of rect() command.

5 Updating Array values using Loops

Similarly, we can use a for loop to add random column height values instead of assigning fixed values in the array.

- Comment out or delete the columns array initialisation command in setup() function. Since we aren't assigning fixed values directly in array, the count cannot have array length. Instead, we need to first decide the array length and assign that value in count variable.

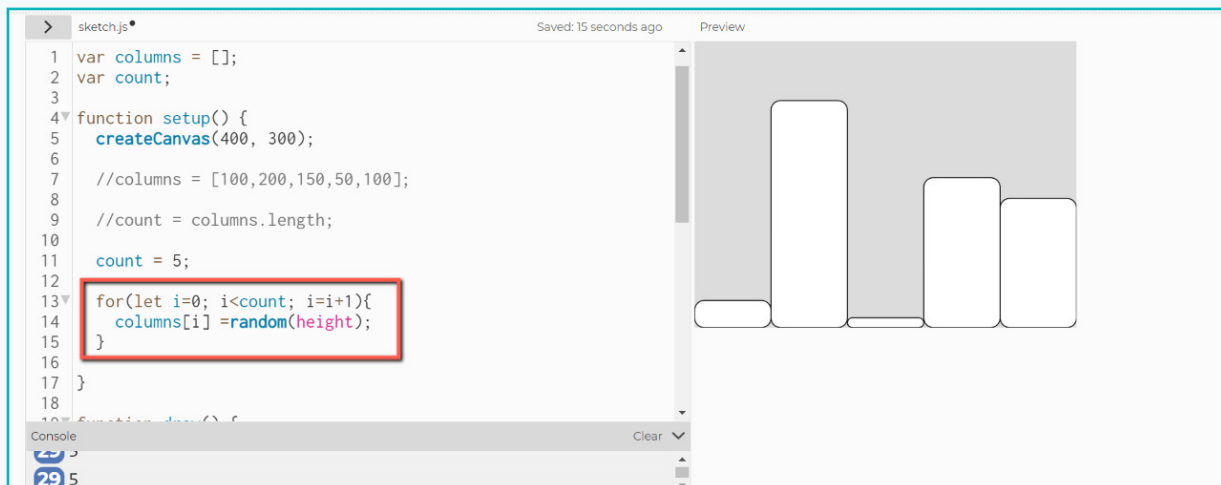
```

1 var columns = [];
2 var count;
3
4 function setup() {
5   createCanvas(400, 300);
6
7   //columns = [100,200,150,50,100];
8
9   //count = columns.length;
10
11   count = 5;
12
13 }
14
15 function draw() {
16
17   console.log(columns.length);
18
19 }

```

Activity

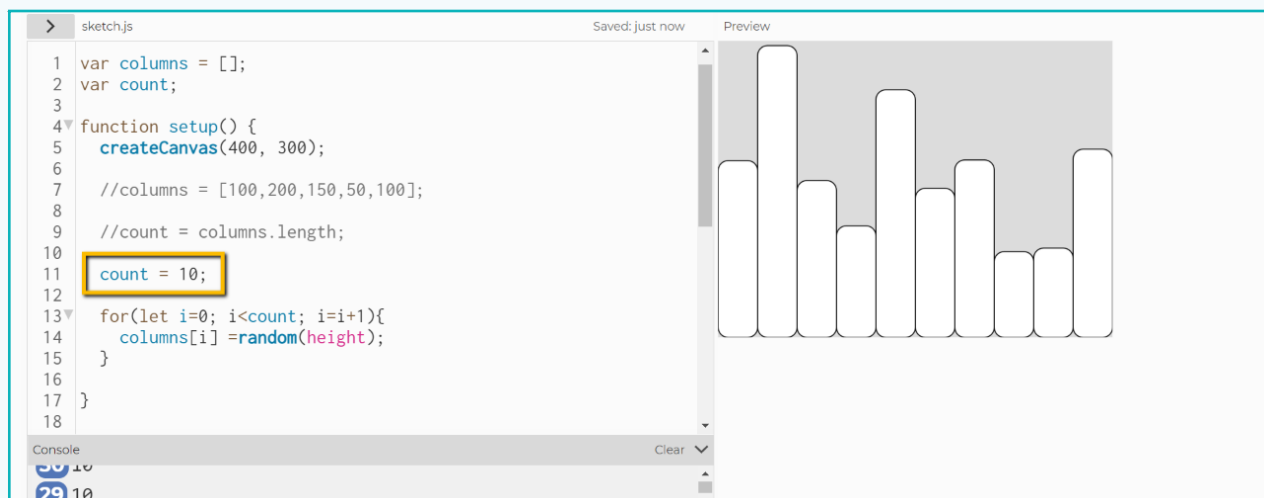
- b. Add for loop with condition “ $i < \text{count}$ ” and increment it by 1.
- c. Assign array values using `columns[i]` in for loop.
- d. Use `random()` command to assign a random value between 0 and canvas height.



```
1 var columns = [];  
2 var count;  
3  
4 function setup() {  
5   createCanvas(400, 300);  
6  
7   //columns = [100,200,150,50,100];  
8  
9   //count = columns.length;  
10  
11   count = 5;  
12  
13   for(let i=0; i<count; i=i+1){  
14     columns[i] = random(height);  
15   }  
16  
17 }  
18  
19 function draw() {  
20   //draw the bars  
21 }
```

The preview window shows a canvas with 5 white bars of varying heights on a gray background. The console shows the value 5.

Now every time we run the code, we get 5 columns of random heights. Now if we just change the value in count variable, we automatically get multiple columns.



```
1 var columns = [];  
2 var count;  
3  
4 function setup() {  
5   createCanvas(400, 300);  
6  
7   //columns = [100,200,150,50,100];  
8  
9   //count = columns.length;  
10  
11   count = 10;  
12  
13   for(let i=0; i<count; i=i+1){  
14     columns[i] = random(height);  
15   }  
16  
17 }  
18  
19 function draw() {  
20   //draw the bars  
21 }
```

The preview window shows a canvas with 10 white bars of varying heights on a gray background. The console shows the value 10.

We have placed columns of random height next to each other. Now, our task is to sort them in ascending order according to their height when a mouse button is clicked.

Exercise

State whether True or False

- 4 For loop is used to perform a repetitive action a specific number of times.

▲ _____

- 5 We can initialise a variable with 'let' keyword instead of 'var' if the variable scope is limited to the block.

▲ _____

Answer in one line.

In, for(let i =0; i < 10; i=i+2) {}

- 6 How many times will the code inside for loop be repeated?

▲ _____

- 7 What will be the value of 'i' after each iteration?

▲ _____

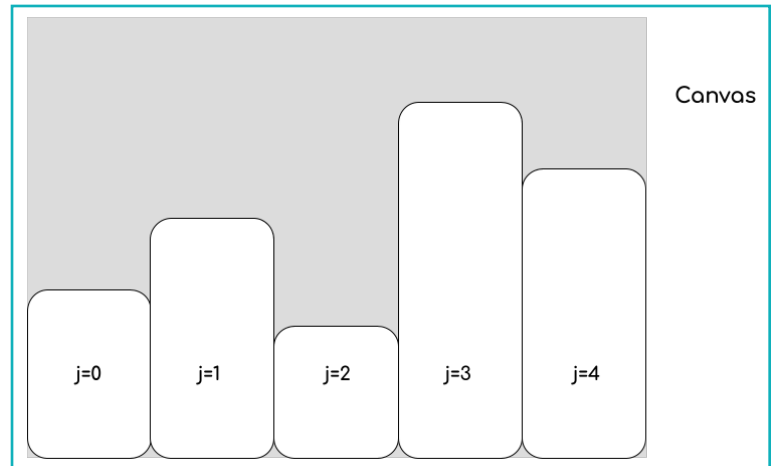
There are many ways to sort a list in a specific order. We will learn the simplest method using a sort algorithm called Bubble sort.

Sorting algorithm

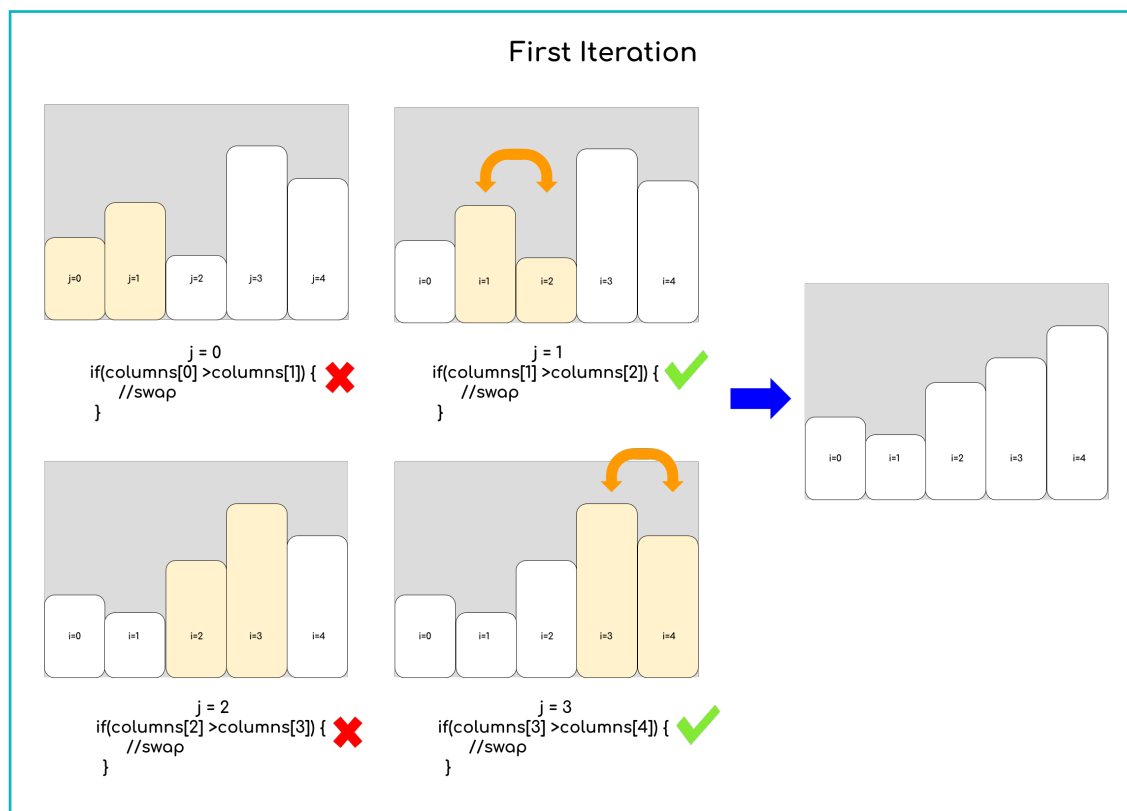
In bubble sort, the process involves iterating through a list that needs to be sorted repeatedly. During each iteration, two consecutive items are compared and swapped if they are in the wrong order. The list is repeatedly passed through until all elements in the list are sorted and no swapping is required.

8. Bubble Sort

For example: Consider 5 columns of random heights.



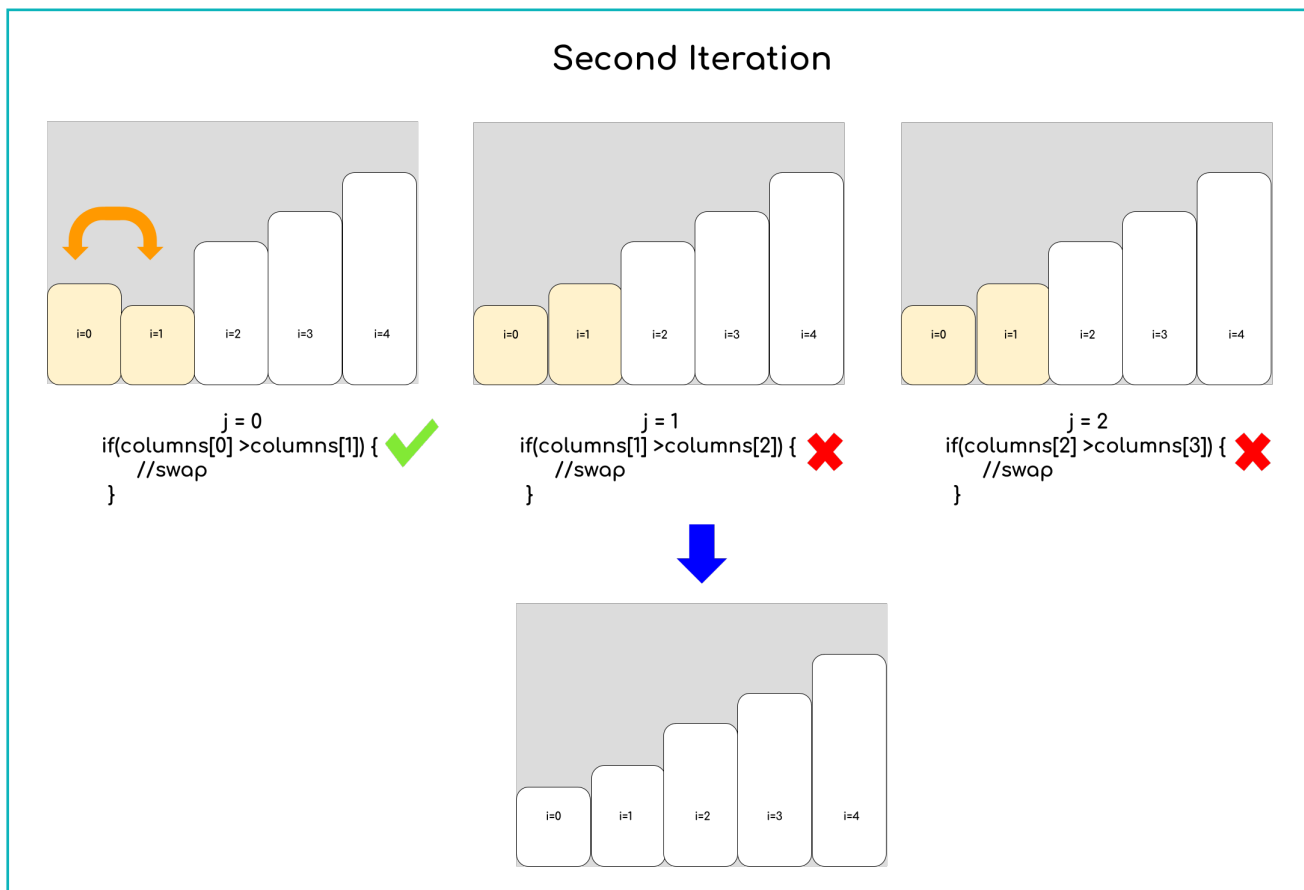
We have to iterate through the array using loop to compare elements. In every iteration, we can compare column heights using condition if (columns[j] > columns[j+1]) for j = 0 to j = 3. If condition is true, we can swap array elements i.e. heights.



In the above diagram, we can see that highest column reaches at the end of the list.

Note that, with this method, the highest columns will move to the end in first round of iteration. Confirm this by drawing different height combinations of columns and performing iteration and swap process as above using pen and paper.

As last column i.e 5th is placed correctly, we have to repeat the procedure to compare first four columns from index 0 to 3. Hence, we can repeat the comparison if $(\text{columns}[j] > \text{columns}[j+1])$ for $j = 0$ to $j = 2$.



Note

The columns are already sorted in order, but this may not be the case all the time. As this algorithm and code should work in case of any order of random columns, we should continue these iterations till the end i.e for $j = 0$ to $j = 1$

Activity

6 Sorting

Note that this is a case of repeating the iterations, and hence, we will use a loop inside another loop. With this concept, let's add code for sorting. Data should be sorted when mouse is clicked, hence add `mousePressed()` function below `draw()` for mouse click event.

```

25 fill(255);
26 // rect(0 , height-columns[0], 80, columns[0], 10);
27 // rect(80 , height-columns[1], 80, columns[1], 10);
28 // rect(160, height-columns[2], 80, columns[2], 10);
29 // rect(240, height-columns[3], 80, columns[3], 10);
30 // rect(320, height-columns[4], 80, columns[4], 10);
31
32 for(let i=0; i < columns.length; i=i+1){
33
34     rect(i*width/count, height-columns[i],width/count,columns[i],10);
35
36 }
37
38
39 function mousePressed(){
40
41
42 }
43

```

The preview shows five white bars of different heights on a gray background. The heights correspond to the values in the `columns` array: 10, 80, 80, 10, 80.

This procedure of comparing columns to each other needs to be repeated 4 times, that is $(\text{count}-1)$ number of times. We will use downward countdown in order to create another loop for column comparison.

- Add for loop to count from `'i = count-1'` with condition that `"i > 0"`. Decrement `i` by 1 to repeat the iteration of swapping columns.

```

29 //rect(240, height - columns[3], 80, columns[3], 10);
30 //rect(320, height - columns[4], 80, columns[4], 10);
31
32 for(let i=0; i < columns.length; i=i+1){
33     rect(i*width/count, height-columns[i],width/count,columns[i],10);
34 }
35
36 }
37
38 function mousePressed() {
39
40     for(let i = count-1; i > 0; i=i-1){
41
42
43     }
44
45 }

```

Another loop needs to be added inside the existing for loop to go through the columns to be sorted in each iteration and compare adjacent column heights.

Activity

- b. Add another for loop inside the existing for loop with $j = 0$ and condition $(j < i)$, incrementing j by 1.



```
> sketch.js Saved: 2 minutes ago
33   rect(i*width/count, height-columns[i],width/count,columns[i],10);
34 }
35
36 }
37
38 function mousePressed() {
39
40   for(let i = count-1; i > 0; i=i-1){
41
42     for(let j=0; j < i; j=j+1) {
43
44     }
45
46   }
47
48 }
49 }
Console Clear
```

A code structure having a loop inside another loop is called as a Nested loop.

Observe the condition $(j < i)$ in the loop. It is because after every iteration, highest column in the group reaches its right position in correct order. Hence, in next iteration we check a smaller number of columns for comparison.

We can now add a condition to compare adjacent columns inside the innermost loop.

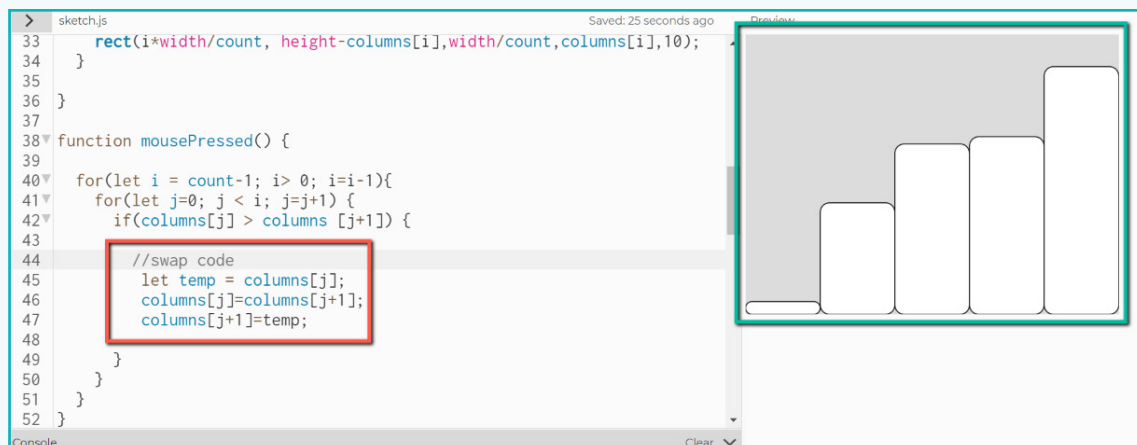
- c. Add if condition to check if adjacent column heights, `columns[j]` and `columns[j+1]` are in the right order. i.e. first column is taller than the second, second is taller than the third and so on.
- d. If it is true, then we can swap them to be in correct order.



```
> sketch.js Saved: just now
31
32 for(let i=0; i < columns.length; i=i+1){
33   rect(i*width/count, height-columns[i],width/count,columns[i],10);
34 }
35
36 }
37
38 function mousePressed() {
39
40   for(let i = count-1; i > 0; i=i-1){
41
42     for(let j=0; j < i; j=j+1) {
43
44       if(columns[j] > columns[j+1]) {
45
46         //swap code
47
48       }
49     }
50
51   }
52
53 }
54 }
55 }
```

Activity

- e. Use swap algorithm and add code to swap values in array elements `columns[j+1]` and `columns[j]` using a temporary variable 'temp'.



```
> sketch.js Saved: 25 seconds ago
33 rect(i*width/count, height-columns[i],width/count,columns[i],10);
34 }
35 }
36 }
37 }
38 function mousePressed() {
39   for(let i = count-1; i > 0; i=i-1){
40     for(let j=0; j < i; j=j+1) {
41       if(columns[j] > columns [j+1]) {
42         //swap code
43         let temp = columns[j];
44         columns[j]=columns[j+1];
45         columns[j+1]=temp;
46       }
47     }
48   }
49 }
50 }
51 }
52 }
```

If we run the code and click the mouse button while the cursor is on the canvas, the columns will sort or rearrange themselves in ascending order. Even if we change the number in count, we can sort multiple columns with the same code.

We learnt how sorting is a necessary task and how we can implement that in programming. We also learnt how arrays and loops help to create short and efficient codes. What other applications can you think of, that uses this code structure?

Exercise

 Tick mark ☒ the correct answer

8 Bubble sort algorithm uses _____ to sort a list in a specific order.

a. nested loop ☐

b. for loop ☐

c. if condition ☐

d. All of them ☐

9 _____ loop means one loop is used within another loop.

a. for ☐

b. nested ☐

c. if ☐

d. None of the above ☐

10 Bubble sort uses _____ algorithm inside nested loops to exchange the values in the array, if two values are not in correct order.

a. sort ☐

b. swap ☐

c. control ☐

d. None of the above ☐

Arrays and loops are effective in sorting data automatically. They help us handle large amounts of data with minimal code.



Tech Flash

- **Array** : A type of data that holds multiple values.
- **Let** : A keyword used to declare a variable instead of 'var' when the variable scope is limited to the code block.
- **Sorting algorithm** : Used to sort a list in a specific required order.
- **Bubble Sort** : The simplest sorting algorithm which works by repeatedly stepping through the list to be sorted, comparing two items at a time, and swapping these two items if they are in the wrong order.
- **Loop** : A code structure used to tell the computer to repeat a specific code block multiple times.
- **For Loop** : A loop statement used to declare an iteration that repeats a set of code instructions to be executed a specific number of times.
- **Nested Loops** : A code structure where we add a loop inside another loop.